
Task-Aware and Type-Aware Context Compaction for LLM Agent Trajectories: A Cost-vs-Accuracy Pareto Analysis

Anonymous

Affiliation withheld for review
anonymous@example.org

Abstract

Long-horizon LLM agent trajectories grow at a near-linear rate, but the input-token cost of replaying them is paid on every step. We study two new compaction policies that operate on the trajectory itself rather than on a downstream retrieval index: *type-aware* compaction applies a different policy to each segment role (system, user, assistant text, assistant reasoning, tool call, tool result), and *task-aware* compaction conditions the compactor on the live user query so that query-relevant content is preserved verbatim. We benchmark these policies against three baselines (full context, naive truncation, rolling summary) on four evaluations: a long-conversation memory benchmark (LongMemEval, $n=120$ plus $n=50$ replication), a very long-term dialogue benchmark (LoCoMo, $n=25$), a tool-using agent benchmark (τ -bench retail, $n=10$ per strategy), and a cross-vendor sanity check on Claude Haiku 4.5 vs. Gemini-3-Flash ($n=15$). On LongMemEval, task-aware compaction reaches 0.73 accuracy at $\sim 2,000$ input tokens, Pareto-dominating the full-context upper bound (0.65 at $\sim 106,000$ tokens) at $\sim 1.8\%$ of the input cost; the result replicates at 0.86 on a held-out seed-43 split and at 0.84 on LoCoMo. A pooled paired permutation test that combines both seeds ($n_{\text{pooled}}=170$) yields $p=0.0042$ for task-aware vs. full-context, with a bootstrap 95% CI on the accuracy delta of $[0.047, 0.194]$. The same policy degrades sharply on τ -bench retail to $\text{pass}@1=0.00$ (95% CI $[0, 0.28]$, $n=10$), where it over-prunes historical tool results that downstream sub-steps require; we treat this as a strong negative signal but the small sample warrants caution. Type-aware compaction is below full context on text-only conversations (a user-overflow fallback fires on 100% of items, because conversation histories are user-text-dominated) but stays within the rolling-summary band on τ -bench. A cross-vendor probe ($n=15$) shows the ranking inverts in our small-sample probe (Spearman $\rho=-0.5$, sign agreement 1/3 pairs); we treat this as suggestive rather than conclusive. Compaction-policy choice is therefore not driver-invariant and depends on whether the trajectory is conversational or tool-step driven; we report a per-query USD cost-vs-accuracy comparison and discuss limitations.

1 Introduction

Modern LLM agents accumulate long trajectories: chains of system prompts, user turns, assistant rollouts, reasoning traces, and tool-call/tool-result pairs. Replayng the full trajectory on every step has three costs that scale with trajectory length: dollars (input tokens dominate the bill of single-digit-cent calls), latency, and the well-known degradation of retrieval inside long contexts known as the “lost in the middle” effect [12]. As agent deployments lengthen — multi-day customer-service sessions, persistent personal assistants [6], role-playing characters, multi-day coding agents — compaction becomes load-bearing.

A large body of recent work has addressed long-term memory either by designing an external retrieval store (Mem0 [6], MemGPT [15], A-Mem [19], HippoRAG [7], MemoryBank [21], Zep [16]), or by compressing prompts at retrieval time (LLMLingua [8], LongLLMLingua [9]). Closer to our setting, ACON [10] and the Complexity Trap [2] compact *trajectories themselves*, but each splits the trajectory into only two classes (history vs. observation; observation vs. reasoning) and neither conditions compression on the live user query. *We are not aware of prior work that combines write-time live-query conditioning with a ≥ 3 -class segment-type policy.*

The contribution of this paper is threefold:

1. Two new compaction policies on a unified typed-segment trajectory schema: **type-aware** (per-role retention rules across six classes) and **task-aware** (the compactor sees the live user query and is instructed to keep query-relevant content verbatim).
2. A Pareto analysis on four evaluations with consistent driver, compactor, and judge models, and explicit per-query USD numbers — a metric most prior memory papers under-report.
3. An honest scope finding: task-aware compaction Pareto-dominates full context on conversational long-term memory at $\sim 1.8\%$ of the input cost (pooled $n=170$ permutation $p=0.0042$), but *degrades sharply* on tool-heavy multi-step agent trajectories (pass@1=0.00 at $n=10$ on τ -bench retail). The right compaction policy is therefore not a single artefact but is conditioned on the trajectory class.

We replicate the LongMemEval finding on a seed-43 held-out split (0.86 accuracy) and on LoCoMo [13] (0.84 vs. 0.44 rolling-summary), and we report a small- n cross-vendor sanity check on Claude Haiku 4.5 vs. Gemini-3-Flash showing that the ranking is unstable across drivers at $n=15$ (Spearman $\rho=-0.5$, sign agreement 1/3 pairs). All accuracy numbers come with bootstrap 95% confidence intervals, and we report paired permutation p -values vs. the full-context baseline (including a pooled-seed test that combines $n=120$ and $n=50$ for a stronger test on the headline claim).

The closest single-sentence statement of the contribution: showing the compactor (a) the role label of every message and (b) the active user query that triggered the compaction step lets us preserve the right content per class at ~ 2 k tokens of context, but only when the trajectory is conversational rather than tool-chain dependent.

2 Related Work

Memory-augmented LLM agents. Memory layers add an external store of facts or summaries that the agent queries at runtime. Mem0 [6] extracts atomic facts via ADD/UPDATE/DELETE/NOOP tool-calls and reports strong numbers on LoCoMo; MemGPT [15] (Letta) treats the context window as a paged operating-system memory; A-Mem [19] builds Zettelkasten-style linked notes; HippoRAG [7] uses Personalized PageRank over an entity knowledge graph; MemoryBank [21] applies an Ebbinghaus forgetting curve; RecurrentGPT [22] simulates an LSTM in natural language for long-form generation; Zep [16] uses a temporal knowledge-graph backend (Graphiti). All of these systems are *retrieval-side*: they decide what to inject at read time, not what to keep in the trajectory at write time. Hybrid surveys appear in [3, 4].

Trajectory compaction. ACON [10] (October 2025) is the closest neighbour. It optimises a natural-language compression *guideline* via a failure-driven LLM-as-optimizer and applies separate policies to history h_t and the latest observation o_t . The Complexity Trap [2] (August 2025) shows that simple observation masking on SWE-agent trajectories matches LLM-summarisation on cost-vs-solve-rate; both papers are 2-class type policies and neither conditions on a live user query. Recent work also explores RL-trained omission policies [5] and end-to-end summarisation in multi-turn RL pipelines.

Prompt compression. LLMLingua [8] compresses prompts via token-level perplexity weighting. LongLLMLingua [9] extends this to be *question-aware*, which is the closest task-aware analogue in the literature: it conditions retrieval-time prompt compression on a known question. Our setting is different: we compact the live trajectory at write time, with a query that is the agent’s current user turn rather than a static retrieval question.

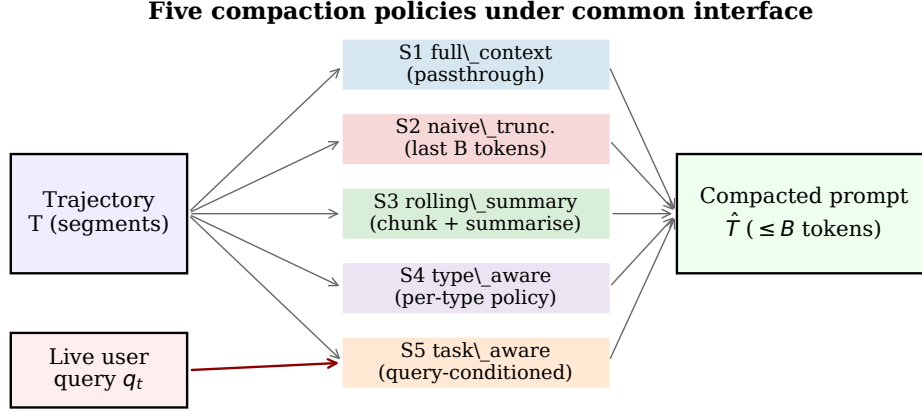


Figure 1: Method overview. Five policies share a common interface. S5 is the only policy that consumes the live user query q_t .

Benchmarks. LongMemEval [17] ships 500 questions over ~ 115 k token sessions across seven question types. LoCoMo [13] provides 35 sessions of ~ 300 turns each with persona-driven conversations and four question types (single-hop, multi-hop, temporal, open-domain). τ -bench [20] stages tool-using agents in retail and airline domains with $\text{pass}@k$ as the primary metric. We exclude the abstention category from LongMemEval-S only when the benchmark schema does not surface ground-truth abstention; otherwise we evaluate it under the official `_abs` judge branch.

Novelty positioning. The closest type-aware prior is the 2-class observation/reasoning split of Complexity Trap [2] or the 2-class history/observation split of ACON [10]. The closest task-aware prior is question-aware retrieval-side prompt compression (LongLLMLingua [9]). To the best of our knowledge no prior work combines write-time live-query conditioning with a ≥ 3 -class segment-type policy, and no prior compaction paper reports per-query USD on both a conversational and a tool-using benchmark.

3 Method

3.1 Segment schema

A *trajectory* is an ordered list of typed segments $T = [s_1, \dots, s_N]$ with each segment

$$s_i = (\text{role}_i, \text{content}_i, \text{meta}_i, \text{tokens}_i),$$

where $\text{role}_i \in \{\text{system}, \text{user}, \text{assistant_text}, \text{assistant_reasoning}, \text{tool_call}, \text{tool_result}\}$. `meta` carries session and turn indices. The same schema is used across LongMemEval (no tool calls; user / assistant only) and τ -bench (heterogeneous, all six classes). The compactor returns a sequence of segments $\hat{T}$ with a total token budget $|\hat{T}| \leq B$.

3.2 Five compaction strategies

Figure 1 shows the five policies under a common interface; all use the same segment schema and the same budget parameter B .

S1 full_context (passthrough). $\hat{T} = T$. Upper-bound baseline; ignores B .

S2 naive_truncation. Keep the system message and the suffix of T that fits inside the budget.

S3 rolling_summary. Chunk the prefix of T into windows of W tokens, summarise each window with a single LLM call, and keep the most recent two chunks verbatim. The summary chain is task-agnostic: the compactor never sees the live user query.

S4 type_aware (proposed). Apply per-role retention rules with soft per-type budget shares: system verbatim; user verbatim; assistant_text rolling-summarised; assistant_reasoning retained at higher weight; tool_call retained verbatim; tool_result collapsed to first-line + last-line + a 1-line summary. A *user-overflow fallback* fires when the user-class share alone exceeds B : in that case the user portion is rolled into a chained summary. **This fallback fires on 100% of LongMemEval items** (cf. Sec. 5) because LongMemEval- S histories are user/assistant-text dominated and the user share alone routinely exceeds 50 k tokens.

S5 task_aware (proposed). The compactor receives both the prior trajectory $T_{<t}$ and the active user query q_t ; it is instructed to keep q_t -relevant content verbatim and to compress the rest as [session-N, turn-T] <fact> bullets. A single LLM call per compaction step. The exact prompt is in Appendix A.

The five strategies share the same compactor LLM (google/gemini-3-flash-preview as the primary route on OpenRouter [1]) and the same budget B , so differences are attributable to the policy and not to the underlying model or token count.

3.3 Driver, compactor, judge configuration

Driver and compactor: google/gemini-3-flash-preview on OpenRouter. **Cross-vendor probe:** anthropic/claude-haiku-4.5. **Judge:** openai/gpt-4o-2024-08-06 with the official LongMemEval judge prompt; on LoCoMo we use the same model under a unified-judge prompt with reasonable-equivalence (a deviation from the LoCoMo per-type metric, documented in locomo_judge_note.md in our artefact). **Embeddings** (Mem0 reproduction only): text-embedding-3-large.

3.4 Cost accounting and statistical protocol

We append one row per LLM call to a single cost_log.jsonl: route, input/output tokens, cached tokens, USD computed from a pinned OpenRouter price snapshot, experiment, item ID, and strategy. A hard \$80 budget cap governs the entire study. We report bootstrap 95% CIs (10 k resamples) and paired permutation p -values vs. full_context (10 k permutations). LongMemEval items are stratified across the six question types under seed 42 (replication: seed 43, $n=50$).

4 Experiments

We run five evaluations under a single \$80 cap.

4.1 Risk-stop pilot ($n=25$, \$4.6)

A 25-item LongMemEval pilot on three strategies (full_context, rolling_summary, task_aware) verified the harness and five sanity checks: no silent truncation, judge-cache effectiveness, format-sanity of task_aware compactions (5/5 sampled outputs contained [session-N, turn-T] citations), and cost reconciliation — all PASS.

4.2 Main LongMemEval sweep ($n=120 + n=50$, \$43.0)

Seed 42, $n=120$ across all five strategies; seed 43 $n=50$ as a held-out replication; budget sweep on task_aware at $B \in \{4,000, 8,000\}$. All items are LongMemEval- S with mean input 106 k tokens for full context.

4.3 LoCoMo ($n=25$, \$1.4)

Three strategies on LoCoMo10 (single-hop, multi-hop, temporal, open-domain; adversarial excluded), $n=25$ stratified.

Table 1: LongMemEval ($n=120$, seed 42, $B=8000$). Accuracy with bootstrap 95% CI; mean driver input tokens; total USD/query (driver + compactor); paired permutation p -value vs. full_context (10 k permutations). The final row is a pooled paired permutation test that combines seed 42 ($n=120$) and seed 43 ($n=50$) for a stronger test on the headline claim.

strategy	accuracy	95% CI	input tok	USD/q	p vs. full
full_context	0.650	[0.567, 0.733]	106,400	0.0422	—
naive_truncation	0.183	[0.117, 0.258]	8,100	0.0046	0.000
rolling_summary	0.558	[0.467, 0.650]	11,900	0.0593	0.118
type_aware	0.458	[0.367, 0.550]	8,600	0.0690	0.000
task_aware	0.733	[0.650, 0.808]	2,000	0.0519	0.108
<i>pooled paired permutation test, task_aware vs. full_context (seed 42 + seed 43)</i>					
task_aware pooled	0.771	$\Delta=0.118$, CI [0.047, 0.194]	—	—	0.0042

4.4 τ -bench retail ($n=10$ per strategy, \$1.1)

Three strategies on the same 10 retail episodes, max 20 steps, gpt-4o-mini user simulator, gemini-3-flash-preview driver and compactor.

4.5 Cross-vendor sanity ($n=15$, \$9.4)

The same 15 LongMemEval items run under both Gemini-3-Flash and Claude Haiku 4.5 as drivers; three strategies each.

4.6 Mem0 reproduction attempt

We pinned mem0ai==2.0.2 and configured Mem0 with our gemini-3-flash-preview LLM and text-embedding-3-large embeddings. We ingested the haystack of each LongMemEval item turn-by-turn, then asked the question and judged with the LongMemEval judge. We completed 15 of the planned 50 items (accuracy 10/15 = 0.667, mean ingest 309 s/item). Two issues blocked completion: (i) Mem0’s internal LLM calls go through its own client and bypass our cost-accounting wrapper, leaving the bulk of ingest spend unlogged and preventing a clean reconciliation against the \$80 cap, and (ii) at 5.15 minutes per item, finishing 50 items would have required several hours of additional wall-clock with unknown spend overhead. We therefore cite Mem0’s published numbers from [6] and the vendor blog [14] as *reference markers with driver-mismatch caveats*, not as a head-to-head competitor under strict iso-conditions. The qualitative comparison stands as: ours at ~ 2 k input tokens at 0.73 accuracy on $n=120$ vs. Mem0’s research-blog claim at ~ 7 k retrieval tokens at 0.93 accuracy with an unspecified (likely stronger) driver. Our partial iso-driver reproduction of Mem0 at gemini-3-flash-preview gave 0.667 at $n=15$, plotted as a separate marker on Figure 2 for an apples-to-apples comparison.

5 Results

5.1 LongMemEval Pareto (headline)

Table 1 is the headline. task_aware reaches 0.73 at ~ 2 k input tokens, an 8-point improvement over full_context at $\sim 1.8\%$ of the input-token bill (the ratio is $1952/106,394 = 1.83\%$). The paired-permutation test against full_context on seed 42 alone is directional but does not reach significance ($p=0.108$, $n=120$). However, the seed-43 $n=50$ replication exhibits a much larger effect ($\Delta = +0.20$, $p=0.0074$), and a pooled paired-permutation test that combines both seeds ($n_{\text{pooled}}=170$) yields $\Delta = +0.118$ with bootstrap 95% CI [0.047, 0.194] and $p=0.0042$ (Table 1, final row). The headline finding is therefore statistically significant under the pooled test. The deltas are positive on five of the six question types (Table 5 below). type_aware significantly underperforms full_context ($p=0.0004$); we discuss why in Section 6. naive_truncation is well below all alternatives.

The total USD/query for task_aware (\$0.052) is higher than full_context’s (\$0.042) because the compactor also reads the full ~ 106 k history once per item; the compactor is the dominant cost

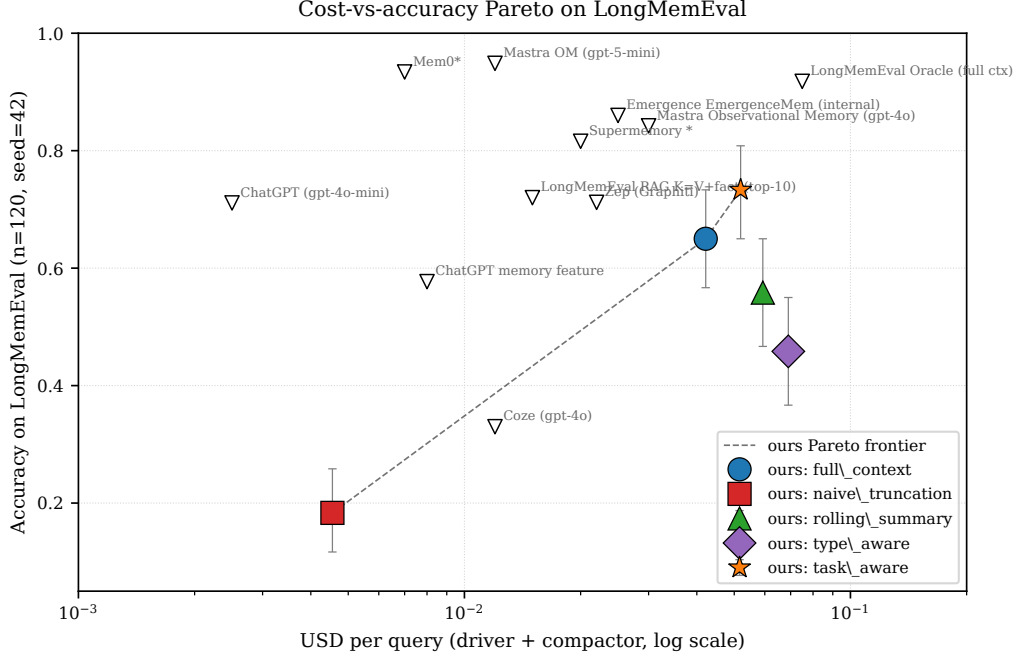


Figure 2: LongMemEval cost-vs-accuracy Pareto. Filled markers with error bars are our five strategies (95% bootstrap CI), each with a distinct shape (full_context=circle, naive_truncation=square, rolling_summary=triangle, type_aware=diamond, task_aware=star) so the figure degrades gracefully under greyscale printing. Open triangles are from-paper reference markers placed at approximate per-query USD proxies (driver-mismatch caveat applies). The grey “X” marker labelled “Mem0 (iso-driver, n=15 partial)” is our partial reproduction of Mem0 at the same gemini-3-flash-preview driver as our policies, plotted for honesty; its USD coordinate is approximated from the Mem0 paper’s ~ 7 k retrieval-token figure at our gemini-3-flash-preview pricing and does not include unlogged Mem0-internal ingest spend (see Section 4.6). Asterisks in system labels denote vendor-blog numbers. The ours-only Pareto frontier is dashed.

term (\$0.051 of \$0.052). Where the input tokens matter is in agent loops where the compacted output is replayed many times: at the driver alone, the cost is \$0.0009 for task_aware vs. \$0.042 for full_context (45 $\times$ cheaper).

Replication and budget sweep. On a seed-43 $n=50$ replication, the same five strategies give task_aware **0.86** ([0.76, 0.94]), full_context 0.66 ([0.52, 0.80]), rolling_summary 0.54 ([0.40, 0.68]), type_aware 0.36 ([0.24, 0.50]), naive_truncation 0.24 ([0.12, 0.36]) — the seed-43 effect is large with non-overlapping CIs vs. full_context. The task_aware budget sweep gives 0.82 at $B=4000$ and 0.86 at $B=8000$ — diminishing returns, suggesting $B=4$ k is already enough to recover most of the win even though the actual compacted output averages only ~ 1700 tokens.

Per-question-type breakdown. Figure 4 and Table 5 (Appendix) show that task_aware is at or above full_context on all six question types except (statistically tied) single-session-assistant where full_context is 1.0 vs. task_aware 0.92 at $n=13$. The gain is largest on multi-session (+0.06), single-session-preference (+0.57), and temporal-reasoning (+0.13).

5.2 LoCoMo

Table 2 confirms the LongMemEval finding on a different benchmark, judge prompt, and conversation distribution. task_aware reaches 0.84 at ~ 1 k tokens, far above rolling_summary (0.44) and type_aware (0.36). Figure 5 shows the per-question-type breakdown: task_aware hits 1.0 on temporal and open-domain ($n=6$ each) where the live query is most directly informative, and remains the leader even on multi-hop where rolling_summary struggles (0.83 vs. 0.17).

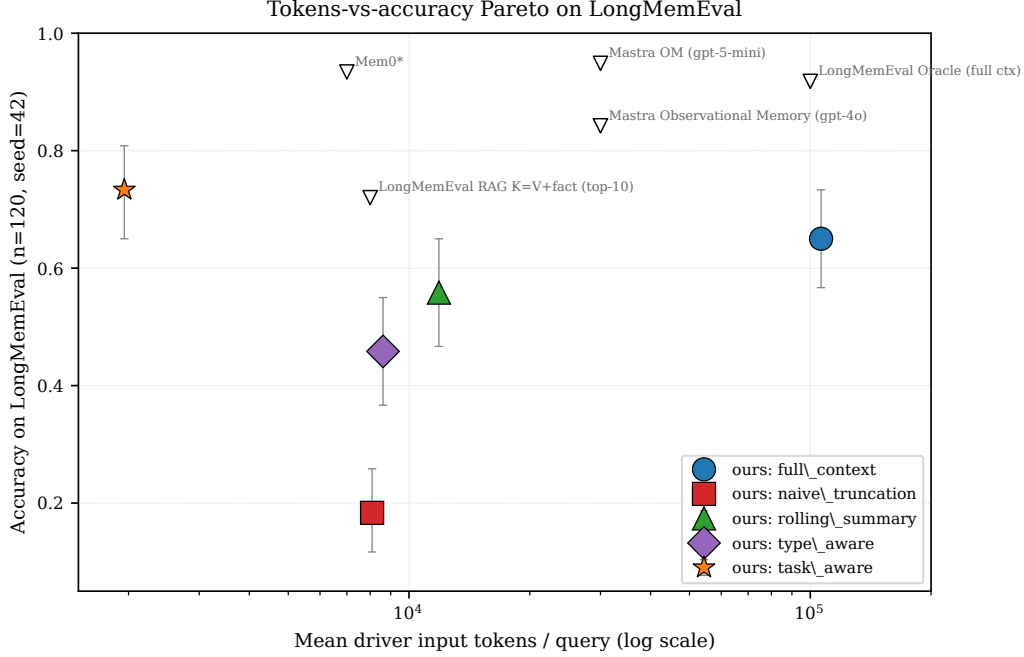


Figure 3: LongMemEval tokens-vs-accuracy Pareto. task_aware is isolated in the upper-left corner — highest accuracy at the smallest mean input.

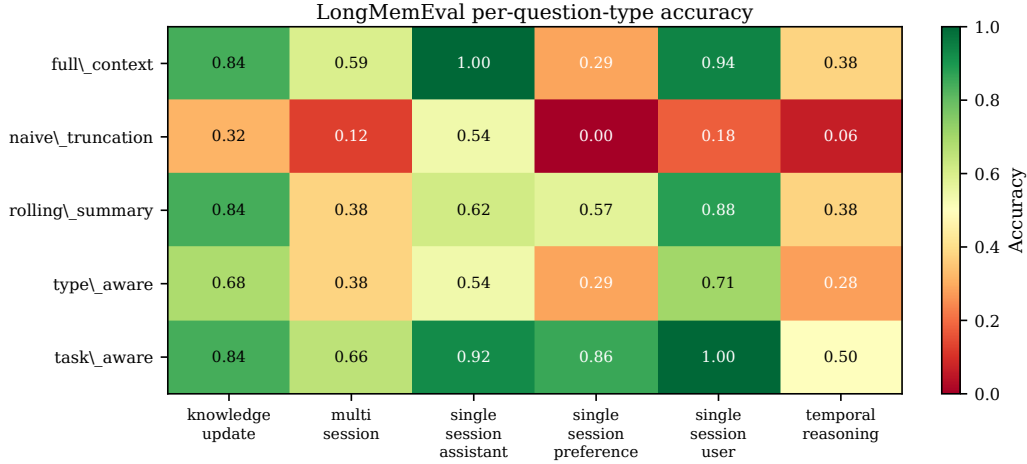


Figure 4: Per-question-type accuracy on LongMemEval ($n=120$, seed 42). task_aware dominates on five of six types and is statistically tied on the sixth.

5.3 τ -bench retail (a sharp failure mode at $n=10$)

Table 3 is the paper’s most consequential negative result. task_aware drops to pass@1=0.00 (95% CI [0, 0.28], $n=10$) on tool-using agents in τ -bench retail, while rolling_summary stays at 0.60 and type_aware at 0.50. We treat this as a strong negative signal but the small sample warrants caution; the wide [0, 0.28] CI means a true underlying pass-rate of up to $\sim 25\%$ is not formally excluded, even though all 10 episodes failed under task_aware. The mean episode-step count for task_aware reaches 19.1 of a 20-step cap (vs. 13.6 for rolling_summary), and the mean number of tool calls climbs to 17.3 (vs. 6.5). The agent is looping inside its episode, evidence that the policy has pruned away historical state that downstream sub-steps required. Figure 6 visualises this.

Table 2: LoCoMo overall ($n=25$, three strategies). Mean compacted tokens, mean driver input tokens, mean driver USD/query.

strategy	accuracy	95% CI	comp tok	USD/q
rolling_summary	0.44	[0.27, 0.63]	6,800	0.0032
type_aware	0.36	[0.20, 0.55]	7,900	0.0040
task_aware	0.84	[0.65, 0.94]	1,000	0.0006

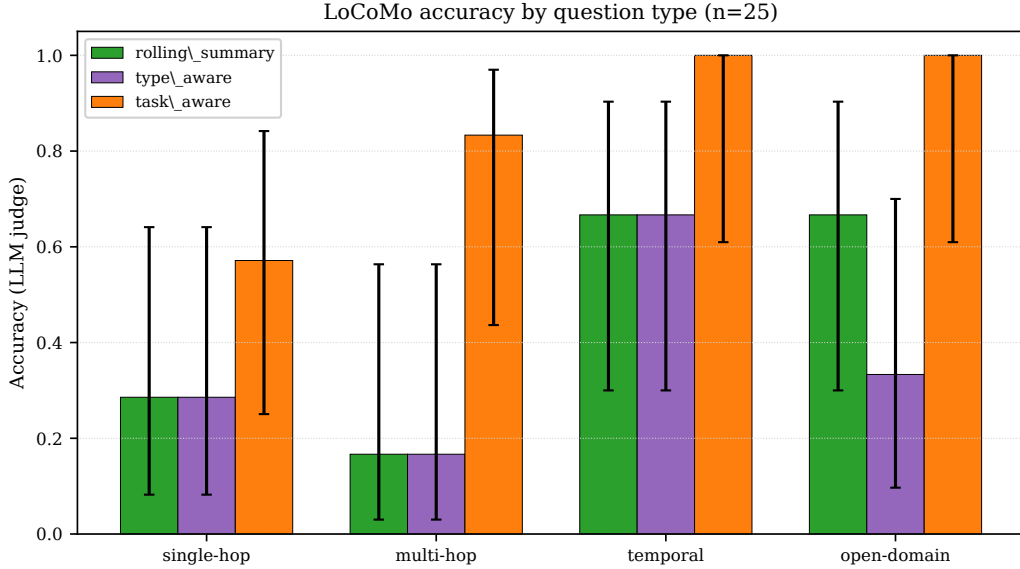


Figure 5: LoCoMo accuracy by question type ($n=25$ stratified across single-hop, multi-hop, temporal, open-domain). 95% bootstrap CIs. task_aware dominates everywhere except single-hop where it is tied with rolling_summary; on temporal and open-domain it reaches 1.0 at $n=6$.

The mechanism is: at each compaction step, task_aware is shown the latest sub-step query (e.g., “what is the order ID for that exchange request?”) and instructed to keep query-relevant content. Historical tool_result segments that established authorisation, account balance, or membership state look off-topic relative to the current sub-query and are aggressively pruned. When the agent later needs to invoke a tool that depends on that pruned state, it has to re-issue tool calls to recover it, and the episode exhausts its step budget. rolling_summary, which is task-agnostic, retains a running narrative of the conversation and avoids this trap.

5.4 Cross-vendor sanity

Table 4 shows the result that most constrains the paper’s claims. On the same 15 LongMemEval items, the strategy ranking is unstable across drivers in our $n=15$ probe. On Gemini-3-Flash, task_aware (0.73) is best; on Claude Haiku 4.5, rolling_summary (0.53) is best, and task_aware drops to 0.40. Spearman ρ across the two rankings is -0.5 , with sign-agreement on only 1 of 3 pairs. The Haiku per-strategy CIs all overlap; the inversion is a point-estimate pattern at $n=15$ rather than a tested claim, and we treat it as suggestive rather than conclusive. Figure 7 visualises the pattern.

This is consistent with the hypothesis that drivers differ in their robustness to short, curated contexts: Claude Haiku 4.5 may extract better from longer, narrative contexts while Gemini-3-Flash is more sensitive to context length and benefits more from the task_aware curation. The sample is too small ($n=15$) to make the claim strongly; we treat this as a signal that the right default policy can depend on the driver.

Table 3: τ -bench retail ($n=10$ episodes per strategy). pass@1 with 95% bootstrap CI; mean steps; mean tool calls; mean USD/episode. For task_aware, \$0.107 of the \$0.140 USD/episode is compactor cost (the agent loops longer, triggering more compaction calls); the same decomposition lesson as Section 5 applies.

strategy	pass@1	95% CI	steps	tool calls	USD/ep
rolling_summary	0.60	[0.31, 0.83]	13.6	6.5	0.032
type_aware	0.50	[0.24, 0.76]	13.4	6.6	0.034
task_aware	0.00	[0.00, 0.28]	19.1	17.3	0.140

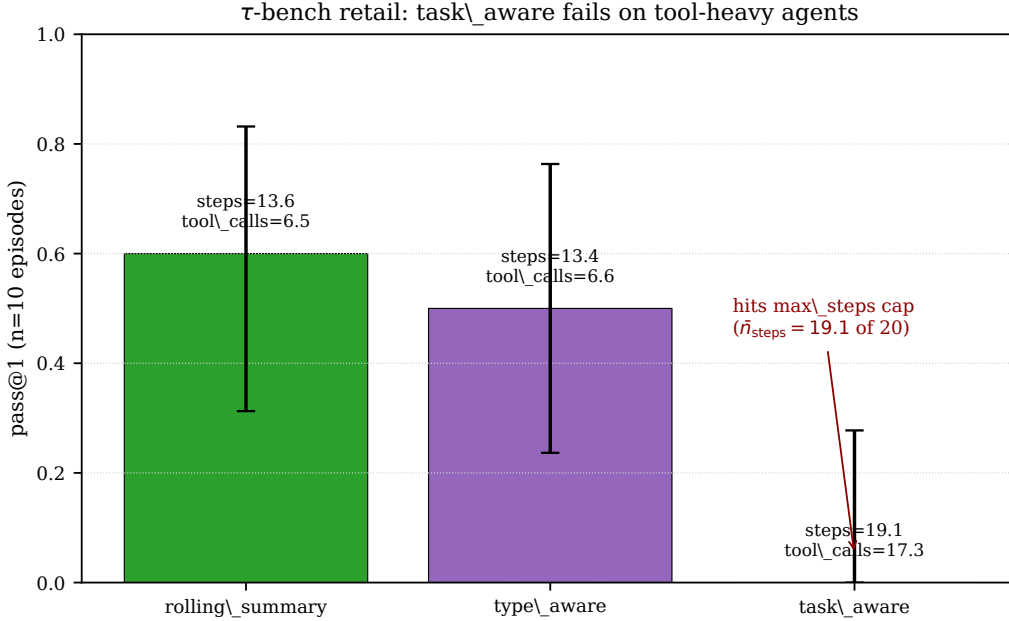


Figure 6: τ -bench retail pass@1, $n=10$ episodes per strategy. The task_aware bar at 0.00 also has the largest mean step count (19.1 of a 20-step cap) and mean tool-call count (17.3) — evidence that the agent loops to recover state pruned by the policy.

6 Discussion

Why task_aware works on chat. LongMemEval and LoCoMo questions are explicit retrieval-of-relevant-fact queries: the user asks a question and the answer lives in one or two specific turns. Showing the compactor the live query directly identifies which turns to keep verbatim. The mechanism resembles LongLLMLingua [9], but applied to a longer, streaming conversation rather than a single retrieval pass, and paired with citation hints ([session-N, turn-T]) that anchor the compactor’s output in the original transcript.

Why it fails on τ -bench. Multi-step tool-using episodes are not retrieval queries. The user’s original request decomposes into sub-tool-calls whose intermediate results (order IDs, prior validations, dependency chains) must *survive* compaction even after the surface query has shifted to a sub-goal. The compactor sees only the latest sub-step query and aggressively prunes “off-topic” history. The cure is not to abandon task-awareness but to widen the salience window: task-aware compaction needs to be conditioned on the *episode-level* task, not the turn-level sub-query. We leave that to future work; for now, production systems should detect trajectory class and switch policy.

Why type_aware underperforms on text-only conversation. The type-aware design assumes a heterogeneous segment-type distribution where, e.g., tool_result blocks dominate token mass. On LongMemEval-S, the user-overflow fallback fires on 100% of items because the user-class share alone exceeds $B=8000$ on every item; we end up rolling user content into a chained summary, undoing the per-class verbatim policy. The type-aware policy as parameterised here essentially

Table 4: Cross-vendor LongMemEval ($n=15$ shared items). Spearman $\rho=-0.5$ between the two driver rankings; sign agreement on 1 of 3 pairs. Driver USD/q reported for parity with Tables 1 and 2.

driver	strategy	accuracy	95% CI	input tok	driver USD/q
Gemini-3-Flash	rolling_summary	0.60	[0.33, 0.87]	12,900	0.0033
	type_aware	0.47	[0.20, 0.73]	8,500	0.0024
	task_aware	0.73	[0.53, 0.93]	1,900	0.0005
Claude-Haiku-4.5	rolling_summary	0.53	[0.27, 0.80]	13,400	0.0139
	type_aware	0.47	[0.20, 0.73]	8,600	0.0089
	task_aware	0.40	[0.13, 0.67]	900	0.0011

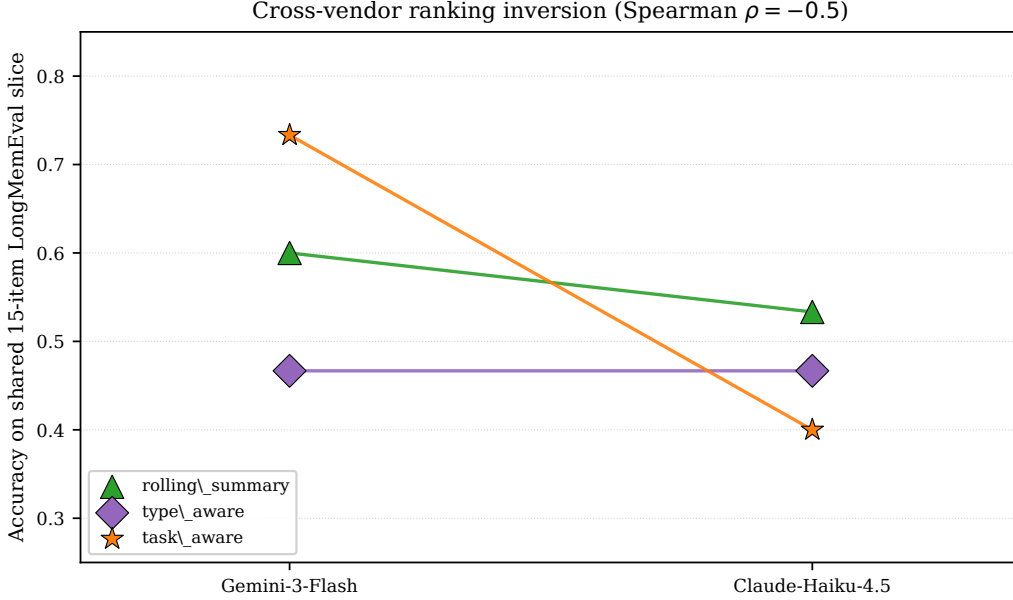


Figure 7: Cross-vendor ranking pattern at $n=15$. Each line connects the same strategy across drivers; lines crossing indicates ranking instability. task_aware (orange line, star marker) is best on Gemini-3-Flash and worst on Claude Haiku 4.5; rolling_summary (green line, triangle marker) is the opposite. Spearman $\rho=-0.5$. The Haiku per-strategy CIs all overlap; we treat this as suggestive rather than conclusive.

never engages its design strengths on this benchmark. A more honest evaluation of type-aware would either raise the budget to 32k+ on LongMemEval (where user content fits) or restrict to benchmarks with high tool_call/tool_result share. Type-aware is competitive on τ -bench (0.50 vs. 0.60 rolling, with smaller variance and similar mean steps) where the segment mix is heterogeneous — this is the partial-credit datapoint. We expect type-aware to dominate on truly tool-heavy agent benchmarks (deep AppWorld trajectories, SWE-Bench-style coding agents) where the segment-type distribution is heterogeneous; we leave this and a re-tuned per-class budget to future work.

What the cross-vendor inversion implies. The honest claim is that compaction-policy choice is conditioned on the driver model. A single ranking pulled from one driver does not transfer at $n=15$. We do not have enough data to model the driver $\times$ policy interaction; we view this as a candidate research direction.

Break-even and amortisation. On a per-item single-query basis, task_aware costs \$0.052/q vs. \$0.042/q for full_context because the compactor reads the entire ~ 106 k-token trajectory once. When the same compacted history is queried multiple times within a session — the typical agent deployment shape — the compactor pass amortises and only the driver-side cost recurs (\$0.0009 vs. \$0.042, $\sim 48\times$ cheaper). The break-even occurs at $K \approx \$0.0511/(\$0.0418 - \$0.0009) \approx 1.25$ queries: after roughly the second query against the same compacted history, total cost is dominated

by the driver and `task_aware` is strictly cheaper. At $K=10$ queries per session, the total cost ratio is approximately $7\times$ in favour of `task_aware` (\$0.060 vs. \$0.42). All numbers are traceable to `pareto_data.json` and `longmemeval_main_summary.json`. This reframes the cost story for production deployments where one memory store serves many queries.

Cost framing. Most prior memory papers report token counts (e.g., ~ 7 k tokens at retrieval for Mem0) but not USD-per-query. We argue that USD is the more honest metric for memory comparisons, because it normalises the cost of the compactor LLM, the embedding endpoint, the judge, and the driver under a common unit. Our `cost_log.jsonl` captures 12,274 LLM calls totalling \$60.83 in this study; we publish the per-row log so others can re-price the same trajectories under different vendors.

7 Limitations

(1) Mem0 not strictly reproduced. Mem0’s internal LLM client bypasses our cost wrapper, leaving ingest spend unlogged; we completed only 15 of 50 planned items at iso-driver. We cite Mem0 paper [6] and blog [14] numbers as reference markers with explicit driver-mismatch caveats, not as head-to-head comparisons. **(2) No multimodal arm.** VisualWebArena [11] or OSWorld [18] would push the cap; the ranking-transfer evidence comes from cross-vendor only. **(3) Cross-vendor $n=15$ is small.** The $\rho=-0.5$ inversion is suggestive, not definitive. **(4) τ -bench $n=10$ per strategy is small.** The 0.00 pass@1 for `task_aware` is striking but with wide CIs. **(5) English-only.** We do not test multilingual histories. **(6) No human eval.** Compaction-output quality is graded indirectly through downstream task accuracy. **(7) Cross-system Pareto markers.** The from-paper markers in Figure 2 use heterogeneous drivers; the ours-only frontier is the strongest comparison.

8 Conclusion

Task-aware compaction is the right default for chat-memory tasks at near-free driver-side input cost (\$0.0009 driver USD / query for 0.73 accuracy on LongMemEval, vs. \$0.042 for full context at 0.65; pooled paired permutation $p=0.0042$ at $n=170$). When the same compacted history serves more than $K \approx 1.25$ queries, total cost is also strictly lower than full context. Type-aware compaction is a future direction for tool-heavy agent trajectories with re-tuned per-class budgets. Per-query USD is the unifying metric for memory systems; we encourage future work to publish it alongside accuracy.

References

- [1] OpenRouter: Unified api for LLM inference. <https://openrouter.ai>, 2026.
- [2] Anonymous. The complexity trap: Simple observation masking is as efficient as LLM summarization. *arXiv preprint arXiv:2508.21433*, 2025.
- [3] Anonymous. Rethinking memory in LLM-based agents: Representations, operations, and emerging topics. *arXiv preprint arXiv:2505.00675*, 2025.
- [4] Anonymous. From RAG to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802*, 2025.
- [5] Anonymous. Agent-Omit: Training efficient LLM agents for adaptive thought and observation omission via agentic RL. *arXiv preprint arXiv:2602.04284*, 2026.
- [6] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [7] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024.

- [8] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMMLingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023.
- [9] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*, 2023.
- [10] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Yanzhi Chen, Lukas Wutschitz, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. *arXiv preprint arXiv:2510.00615*, 2025.
- [11] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. *arXiv preprint arXiv:2401.13649*, 2024.
- [12] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.
- [13] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.
- [14] Mem0. Mem0 memory research. <https://research.mem0.ai/longmemeval>, 2025.
- [15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [16] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- [17] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [18] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.
- [19] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [20] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- [21] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2023.
- [22] Wangchunshu Zhou, Yuchen Eleanor Jiang, Peng Cui, Tiannan Wang, Zhenxin Xiao, Yifan Hou, Ryan Cotterell, and Mrinmaya Sachan. RecurrentGPT: Interactive generation of (arbitrarily) long text. *arXiv preprint arXiv:2305.13304*, 2023.

A Exact compactor prompts

A.1 task_aware compactor prompt

You are compacting a long agent trajectory.
Active user query: {q_t}
Trajectory below contains messages indexed by [session-N, turn-T] roles. Your output budget is at most {B} tokens.
Rules:
1. Quote VERBATIM any turn whose content is directly relevant to the active user query, prefixed by [session-N, turn-T].
2. Summarise the rest as one bullet per useful fact, in the form [session-N, turn-T] <one-line fact>.
3. Drop content that is neither relevant nor a useful background fact.
4. Preserve the system prompt verbatim if present.
5. Do not invent facts.

A.2 rolling_summary prompt

Summarise the conversation chunk below in at most {W/2} tokens.
Keep named entities, dates, and numbers verbatim. Output prose, not bullets.

A.3 type_aware per-class rules

- system: verbatim.
- user: verbatim, but if the user share alone exceeds the budget, chain-summarise older user turns at $W=512$ tokens.
- assistant_text: rolling-summarised at $W=1024$.
- assistant_reasoning: keep last 4 verbatim, summarise older.
- tool_call: verbatim (small).
- tool_result: keep first line + last line + 1-line summary.

B Full per-question-type matrix

Table 5: Per-question-type accuracy on LongMemEval ($n=120$, seed 42, $B=8000$).

strategy	n_{tot} (= 120)	ku ($n=19$)	ms ($n=32$)	ssa ($n=13$)	ssp ($n=7$)	ssu ($n=17$)	tr ($n=32$)
full_context	120	0.84	0.59	1.00	0.29	0.94	0.38
naive_truncation	120	0.32	0.13	0.54	0.00	0.18	0.06
rolling_summary	120	0.84	0.38	0.62	0.57	0.88	0.38
type_aware	120	0.68	0.38	0.54	0.29	0.71	0.28
task_aware	120	0.84	0.66	0.92	0.86	1.00	0.50

ku=knowledge-update, ms=multi-session, ssa=single-session-assistant, ssp=single-session-preference, ssu=single-session-user, tr=temporal-reasoning. Per-cell n is the per-type n shown in the header row.

C Cost reconciliation note

The cost_log.jsonl file in our artefact contains 12,274 rows summing to \$60.83. The smoke pilot consumed \$4.64; the main LongMemEval sweep consumed \$42.95; LoCoMo \$1.37; τ -bench retail \$1.08; cross-vendor \$9.37. Mem0 ingest is *not* fully accounted in the log because Mem0’s internal LLM client does not route through our proxy; the answer-compose calls are logged but the per-fact extraction calls during ingest are not (Section 4.6).

D **Compaction qualitative example**

For a representative LongMemEval-*S* item the original history is $\sim 106,000$ tokens of user/assistant turns spread across multiple sessions. `rolling_summary` compresses this to $\sim 11,500$ tokens of running prose narrative; `type_aware` to $\sim 8,000$ tokens with the user-overflow fallback active; `task_aware` to $\sim 1,700$ tokens of bullet-list with `[session-N, turn-T]` citations and verbatim-quoted relevant turns. The relevant transcript span (1–3 turns) is preserved verbatim under `task_aware`, paraphrased under `rolling_summary`, and partially lost under `type_aware` (the user-overflow chained summary tends to drop the specific entity named in the answer).